In this chapter, we look at common problems such as slow query and ingestion speeds to understand the reasons behind them. Because Elasticsearch is a complex distributed architecture, there are several places to look for fixes. We discuss the most obvious and commonly applied solutions in this chapter.

We also look at problems arising from unstable clusters and troubleshoot them with the help of cluster health, cluster allocation, and node APIs. Then we discuss how Elasticsearch sets memory and disk usage thresholds to keep the cluster alive and kicking. Let's begin by seeing why our search queries do not yield faster results and what troubleshooting options are available.

> **NOTE** As mentioned in chapter 14, similar to administration of Elasticsearch, performance tuning is an advanced topic that requires expert hands on the deck. Before tweaking your app for performance, consult the experts, read the documentation, and experiment in your lab environment. This chapter presents a helicopter view of the performance landscape.

15.1 Search and speed problems

Although Elasticsearch is a near-real-time search engine, we must carefully tame the beast to make sure it works as expected in various scenarios. Over time, Elasticsearch's performance may degrade if not architected well with future data requirements in mind and if not constantly maintained. Degradation of the server's performance is detrimental to overall cluster health, which affects search query and indexing performance.

The major problems that Elasticsearch users report most commonly are slow serving of search queries and slow indexing speeds. A unique selling point for Elasticsearch is its blazingly fast queries. However, we can't expect the out-of-the-box solution to fit all our needs. A multitude of variables such as sensible distribution of nodes, shards, and replicas depending on memory management and hardware availability allow us to create a healthy cluster as we prepare and productionize the infrastructure and applications.

Often, Elasticsearch is incorrectly set up for the search scenario at hand. For example, some applications are intensive on the search side, whereas others need to serve less-frequent searches.

15.1.1 Modern hardware

Elasticsearch uses Lucene under the hood for indexing and storing data. Data is stored in the filesystem, and although Lucene does this efficiently, additional help such as providing solid state drives (SSDs) rather than hard disk drives (HDDs), allocating abundant memory, assisting in infrequent merges, and so on goes a long way.

Each node consists of shards and replicas as well as other cluster data, including internal data structures that Elasticsearch maintains. Because Elasticsearch is developed in Java, allocating sizable heap memory overall helps with the smooth running of the application. The *heap* is the memory location where new objects are stored in the young generation space. As this space fills up, any objects surviving minor garbage